

Programação gráfica usando o Gloss

Laboratórios de Informática / Programação I
LEI / LCC

Para construir a interface gráfica do projeto far-se-á uso da biblioteca Gloss. O Gloss é uma biblioteca Haskell minimalista para a criação de gráficos e animações 2D. Como tal, é ideal para a prototipagem de pequenos jogos. A documentação da API da biblioteca encontra-se disponível em <https://hackage.haskell.org/package/gloss>.

Instalação

O Gloss pode ser instalado através do utilitário `cabal`, o gestor de bibliotecas Haskell. Em vez de instalar o Gloss diretamente, sugere-se que acrescentem a biblioteca como dependência nos vossos projetos `.cabal`:

```
build-depends: gloss
```

A instalação será automática. Nos vossos programas Haskell, devem acrescentar o `import Graphics.Gloss` para utilizar a biblioteca.

OpenGL

A biblioteca Gloss depende internamente da biblioteca OpenGL para geração de gráficos. Consoante o seu sistema operativo, garanta que tenha instalado:

- Ubuntu: `sudo apt install freeglut3-dev`
- Mac: `brew install glfw`

Criação de gráficos 2D

O tipo central da biblioteca Gloss é o tipo `Picture`. Este permite criar uma figura 2D usando segmentos de recta, círculos, polígonos, ou até *bitmaps* lidos de um ficheiro. A cada um destes diferentes tipos de figura correspondem diferentes construtores do tipo `Picture` (e.g. o construtor `Circle` para um círculo - ver [documentação](#) para consultar listagem completa dos construtores). Por exemplo, o valor `circulo` definido abaixo representa um círculo de raio 50 centrado na posição (0,0).

```
circulo :: Picture
circulo = Circle 50
```

Certos construtores do tipo `Picture` não representam propriamente figuras, mas antes transformações sobre sub-figuras. Por exemplo, o construtor `Translate :: Float -> Float`

`-> Picture -> Picture` permite reposicionar uma figura efetuando uma translação das coordenadas. Assim, para posicionar o círculo atrás definido num outro ponto que não a origem bastaria fazer algo como:

```
outroCirculo :: Picture
outroCirculo = Translate (-40) 30 circulo
```

Outras transformações possíveis são `Scale`, `Rotate` e `Color`. Por último, podemos ainda produzir uma figura agregando outras figuras usando o construtor `Pictures :: [Picture] -> Picture`, que recebe uma lista de figuras para serem desenhadas sequencialmente (note que essas figuras se podem sobrepôr entre si). Segue-se um exemplo onde se explora essa possibilidade juntamente com outras transformações:

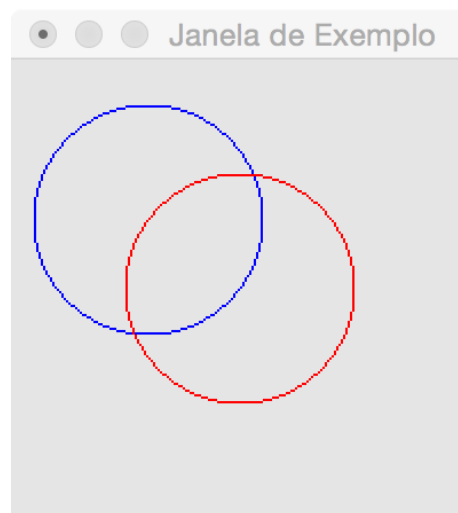
```
circuloVermelho = Color red circulo
circuloAzul = Color blue outroCirculo
circulos = Pictures [circuloVermelho, circuloAzul]
```

Naturalmente que o objetivo de definir figuras como valores do tipo `Picture` é podermos visualizá-las no ecrã. Para tal temos de criar uma janela Gloss onde será desenhado o conteúdo da figura. O fragmento de código que se segue permite visualizar a figura `circulos` definida atrás:

```
window :: Display
window = InWindow
    "Janela de Exemplo" -- título da janela
    (200,200)           -- dimensão da janela
    (10,10)             -- posição no ecrã

background :: Color
background = greyN 0.8

main :: IO ()
main = display window background circulos
```



Para correr o programa basta compilar o ficheiro Haskell usando o `ghc` e correr o executável. De notar que a convenção no Gloss é que a posição com coordenadas (0,0) é o centro da janela. Assim, o resultado obtido será a janela apresentada em cima.¹

¹ Dependendo do sistema operativo e instalação do OpenGL, pode também ser capaz de interpretar uma aplicação gloss com o interpretador `ghci` utilizando a opção `-fno-ghci-sandbox`.

Programação de jogos

Para além da visualização de gráficos 2D, a biblioteca Gloss permite criar facilmente jogos simples usando a função `play` da biblioteca `Graphics.Gloss.Interface.Pure.Game`. O seu tipo é descrito de seguida:

```
play ::
  Display -- definição da janela
  -> Color -- cor do fundo da janela
  -> Int -- número de passos de simulação por segundo (frame rate)
  -> world -- estado inicial
  -> (world -> Picture) -- função que converte um estado num valor do tipo
Picture
  -> (Event -> world -> world) -- função que reage a um evento calculando
o próximo estado
  -> (Float -> world -> world) -- função que altera o estado em função do
tempo
-> IO ()
```

Para usar esta função é necessário começar por definir um novo tipo `Estado` que representa todo o estado do seu jogo. Imagine por exemplo que o estado apenas indica a posição atual de um objeto.

```
type Estado = (Float,Float)
```

Depois é necessário definir qual o estado inicial do jogo, e como é que um determinado estado do jogo será visualizado com gráficos 2D, ou seja, como se converte para um valor do tipo `Picture`. No nosso caso, vamos assumir que o nosso estado inicial é a posição (0,0) e que em cada instante de tempo apenas desenhamos um polígono na posição dada pelo estado atual.

```
estadoInicial :: Estado
estadoInicial = (0,0)

desenhaEstado :: Estado -> Picture
desenhaEstado (x,y) = Translate x y poligno
  where
    poligno :: Picture
    poligno = Polygon [(0,0),(10,0),(10,10),(0,10),(0,0)]
```

Para implementar a reação a eventos, nomeadamente o pressionar das teclas, é necessário implementar uma função que, dado um valor do tipo `Event` (definido em `Graphics.Gloss.Interface.Pure.Game`) e um estado do jogo, gera o novo estado do jogo. No nosso exemplo, vamos apenas alterar o estado conforme o utilizador carrega nas teclas “left”, “right”, “up”, and “down”.

```
reageEvento :: Event -> Estado -> Estado
reageEvento (EventKey (SpecialKey KeyUp)    Down _ _) (x,y) = (x,y+5)
reageEvento (EventKey (SpecialKey KeyDown)  Down _ _) (x,y) = (x,y-5)
reageEvento (EventKey (SpecialKey KeyLeft)   Down _ _) (x,y) = (x-5,y)
reageEvento (EventKey (SpecialKey KeyRight)  Down _ _) (x,y) = (x+5,y)
reageEvento _ s = s -- ignora qualquer outro evento
```

Finalmente, é necessário definir a seguinte função que altera o estado do jogo em consequência da passagem do tempo. Se o jogo estiver a funcionar a uma frame rate `fr`, o parâmetro `n` será sempre `1/fromIntegral fr`. Vamos assumir para o nosso exemplo que a cada instante de tempo a posição actual é actualizada da seguinte forma:

```
reageTempo :: Float -> Estado -> Estado
reageTempo n (x,y) = (x,y-0.1)
```

Para colocar todas estas peças a funcionar em conjunto basta definir um programa como o que se segue:

```
fr :: Int
fr = 50

dm :: Display
dm = InWindow "Novo Jogo" (400, 400) (0, 0)

main :: IO ()
main = do play dm          -- janela onde irá correr o jogo
          (greyN 0.5)      -- cor do fundo da janela
          fr                -- frame rate
          estadoInicial    -- estado inicial
          desenhaEstado    -- desenha o estado do jogo
          reageEvento       -- reage a um evento
          reageTempo        -- reage ao passar do tempo
```

Inclusão de imagens no jogo

É possível carregar ficheiros de imagens externos no formato *bitmap* (com extensão *bmp*). Para tal, pode usar a função `loadBMP :: FilePath -> IO Picture` disponibilizada pelo módulo `Graphics.Gloss.Data.Bitmap`. Como esta é uma função de I/O, deve ser executada diretamente na função `main` do jogo, devendo os gráficos ser incluídos no estado do jogo:

```
type EstadoGloss = (Estado, Picture)

...

main :: IO ()
main = do p <- loadBMP "imagem.bmp"
  play dm          -- janela onde irá correr o jogo
    (greyN 0.5)    -- cor do fundo da janela
    fr             -- frame rate
    (estadoGlossInicial p) -- estado inicial
    desenhaEstadoGloss -- desenha o estado do jogo
    reageEventoGloss -- reage a um evento
    reageTempoGloss  -- reage ao passar do tempo
```

Uma dica adicional é que a biblioteca `Gloss`, por definição, apenas suporta ficheiros *bitmap* não comprimidos. Em sistemas Unix, pode-se utilizar a ferramenta `convert` distribuída com o `ImageMagick` para descomprimir um ficheiro *bitmap*:

```
convert compressed.bmp -compress None decompressed.bmp
```

Alternativamente, pode-se também utilizar o pacote `gloss-juicy` que suporta ficheiros de imagens genéricos, alternativamente carregados da seguinte forma:

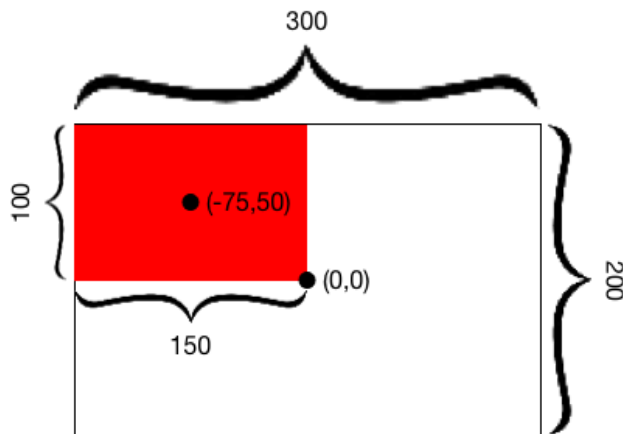
```
...
import Graphics.Gloss.Juicy
...
do
  Just img1 <- loadJuicy "imagem.jpg"
  Just img2 <- loadJuicy "imagem.png"
...
```

Para utilizar esta biblioteca, lembrem-se de a adicionar às dependências do vosso projeto `.cabal`:

```
build-depends: gloss-juicy
```

Gráficos 2D declarativos

A biblioteca Gloss simplifica enormemente o desenvolvimento de aplicações gráficas 2D. No entanto, na sua simplicidade a biblioteca não oferece directamente mecanismos de suporte à programação modular de interfaces gráficas, o que se pode tornar um problema para aplicações mais complexas, ou até mesmo de média dimensão como o jogo proposto para este projeto prático. Considerem que desejamos desenhar um retângulo vermelho ocupando $\frac{1}{4}$ da área da janela e alinhado no quadrante superior esquerdo da janela, como na figura abaixo.



Assumindo que a janela tem dimensão 300x200, a figura pode ser desenhada com o código:

```
rect :: Picture
rect = Translate (-75) (50) $ Color red $ rectangleSolid 150 100
```

Para compreender a relação entre a figura e o código, relembre que:

- O retângulo deve ter metade das dimensões da janela;
- O centro da janela corresponde à posição (0,0);
- O retângulo é desenhado por defeito com centro na posição (0,0);
- Para posicionar o retângulo no quadrante superior esquerdo, é preciso movê-lo em precisamente metade do seu comprimento para a esquerda (coordenadas negativas no eixo do X) e metade da sua altura para cima (coordenadas positivas no eixo do Y).

Para desenhar figuras mais complexas, com um maior número de elementos será portanto necessário fazer bastante aritmética de posições, em função dos tamanhos concretos da janela e dos elementos a desenhar.

Como complemento ao Gloss tradicional, sugere-se que experimentem a biblioteca `gloss-relative`, que propõe um novo tipo de dados `Frame` (de *picture frame*, ou seja, moldura) cuja intuição é construir gráficos decompondo modularmente a janela principal em sub-janelas de menor dimensão. Ver [documentação](#) para uma apresentação mais detalhada. Por exemplo, a `Picture` desenhada acima pode ser descrita de forma mais sucinta pelo seguinte `Frame`:

```
...
import Graphics.Gloss.Relative
...
rect :: Frame
rect = Zoom 0.5 0.5 alignTopLeft (solid red)
```

Note que, desta vez, as dimensões concretas da janela não foram necessárias para escrever o código. O construtor `Zoom` define uma janela com metade da dimensão da janela original, alinhada no quadrante superior esquerdo com `alignTopLeft`. A função `solid` preenche um `Frame` todo com a mesma cor.

Para utilizar esta biblioteca, lembrem-se garantir que têm acesso à versão mais recente (executando o comando `cabal update`) e de a adicionar às dependências do vosso projeto `.cabal`:

```
build-depends: gloss-relative
```

Templates de código

Pode descarregar e estudar alguns exemplos completos de programas utilizando o Gloss, incluindo os utilizados neste tutorial, no seguinte [link](#). Encontrará os seguintes ficheiros:

1. `Gloss1.hs`, que contém o primeiro exemplo de gráficos estáticos deste tutorial. Pode correr com `cabal run gloss1`.
2. `Gloss2.hs`, que contém um exemplo de gráficos dinâmicos com reação ao tempo. Neste caso, um círculo com uma dada velocidade inicial e que sofre o efeito da gravidade. Pode correr com `cabal run gloss2`.
3. `Gloss3.hs`, que contém um exemplo de gráficos dinâmicos com reação a eventos. Neste caso, um retângulo que se desloca com as setas do teclado. Pode correr com `cabal run gloss3`.
4. `Gloss4.hs`, que contém um exemplo de como carregar imagens. Neste caso, uma bola que cai com o tempo e pode ser movida com o teclado. Pode correr com `cabal run gloss4`.
5. `Gloss5.hs`, que contém o mesmo exemplo que o anterior, mas utilizando uma imagem com fundo transparente. Neste caso, utilizando a biblioteca `gloss-juicy`. Pode correr com `cabal run gloss5`.
6. `Gloss6.hs`, que contém um exemplo de um jogo de damas, exemplificando a construção de um tabuleiro de jogo e de uma peça arrastável com o rato. Neste caso, utilizando a biblioteca `gloss-relative`. Pode correr com `cabal run gloss6`.